

Accelerated Stable Differentiation: A Vectorized Approach to Pricing Auto-Callables in the Log-Space with Automatic Adjoint Differentiation

Akshat Kumar

February 2026

Abstract

In this paper, we will study the pricing of Auto-callable notes, a special kind of option that can terminate early due to a barrier condition on the underlying. We will specifically be studying Auto-Callable notes with only one underlier for this paper. Standard Monte-Carlo algorithms work reasonably well in terms of general pricing, but they are not stable with respect to differentiation. This makes the calculation of sensitivities a challenging task, especially in the case of second-order sensitivities like Vanna. In addition, the slow convergence rate of $O(N^{-1/2})$ necessitates a high number of simulations (generally $\approx 10^6$) before traditional Monte Carlo methods converge to a reasonable value.

In this paper, we will introduce an approach that differs from the standard regularized differentiation schemes. We will begin by building off the idea of One-step survival as first discussed by Glasserman and Staum in [GS01]. Much of the notation and conventions used here will come from [Alm+13] and their interpretation of One-step survival for Auto-Callable Notes. We propose a framework that integrates One-Step Survival (OSS) estimation within a vectorized log-space architecture, implemented via Adjoint Algorithmic Differentiation (AAD), which is achieved by rewriting the estimator as a differentiable computational graph. This approach has three main benefits over the traditional Monte-Carlo approach: enhanced stability of first and second order sensitivities, reduced variance in pricing, and computational efficiency through leveraging hardware-accelerated vectorization. There is also an added implicit flexibility in calculating more advanced sensitivities than the traditional delta, gamma, and theta. We find a decrease of several orders of magnitude for the variances of Greeks like Gamma, Delta, and Vega.

1 Introduction

The Application of Monte-Carlo based approaches to options pricing goes back to 1977 with the paper by Boyle [Boy77], which came only four years after the seminal work by Black-Scholes. Even in this first attempt, Boyle clearly recognized and expressed the insufficiency of the traditional Monte Carlo approach in regard to options pricing. This general need comes from the non-differentiable nature of options pricing structures. Since options pricing structures are generally piece-wise linear, even in the case of non-exotic instruments, there is a fundamental discontinuity in first-order derivatives that creates unstable behavior in sensitivities and the evaluations themselves. Analytically, payout functions are generally not differentiable and are often not even continuous.

Boyle's initial work focused on the use of antithetical and control variates to deal with the issue of stability. The industry has since moved on to complex path-dependent structures. Express certificates and other Autocallable instruments exacerbate these issues by introducing digital barriers. These barriers act as discontinuities where a small change in the underlier's price causes a jump in the value of the contract.

Structured products, Auto-Callable notes in particular, have seen a massive jump in market share in the European Union, with Germany and Slovakia being notable examples. "Given the tax and regulatory framework, the focus was primarily on express certificates utilizing the autocall feature," Miroslav Dobsovic, country manager at Cyrrus Slovakia, told SRP [Pro25]. Given the prevalence of these instruments in the European markets, having a clear, descriptive set of statistics is of utmost importance to those on the buy side, sell side, and regulatory bodies. It is worth noting that of these securities, the express certificate is especially popular; with most of these express certificates having a single underlier [Alm+13].

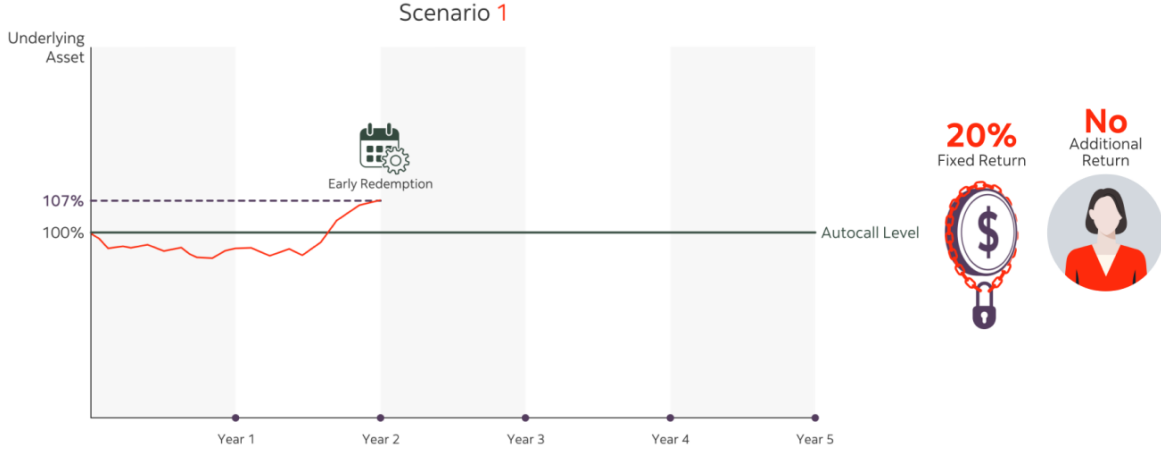


Figure 1: Visual of an Auto-Call

The Auto-Callable note is defined by its main feature, the ability for it to terminate before reaching maturity upon reaching some barrier condition on any one of a given set of pre-determined observation dates. If it does not cross this barrier on any of the given dates, then it is paid out according to a payoff function. The anatomy of an Auto-Callable note generally consists of a fixed-income bond and a menagerie of embedded derivatives. This auto-call feature introduces even more challenges for traditional monte-carlo based pricing methods and makes sensitivities even more unstable. In the next section, we will lay some groundwork in terms of notation and move on to a derivation of the model and evidence of its efficacy and stability.

2 Abstraction and Log-Space Path Generation

2.1 Regarding measure

One point of interest with respect to the model in this paper. We will be working within the $\mathbf{Q}$ measure as opposed to the $\mathbf{P}$. This is a trivial change in this case since our underlier is modeled by Geometric Brownian Motion, but it is something to take note of. μ in this case will be $r - b$ where r is the risk-free interest rate and b is the dividend. Sticking to the $\mathbf{Q}$ measure allows for a more subtle study of volatility and initial price and is the industry standard for pricing. If you seek to forecast instead of price, Girsanov's theorem provides an easy framework for moving between the two measures.

2.2 On Generating Stock Price Paths

Let S_t be a stochastic variable representing the price of the underlying stock at time t . We will operate under the assumption that the stock price evolves according to the Geometric Brownian Motion stochastic differential equation. That is

$$dS_t = \mu S_t dt + \sigma S_t dW_t$$

Now, if we define $X_t := \ln(\frac{S_t}{S_0})$, we find that the variable X_t evolves according to an Arithmetic Brownian Motion stochastic differential equation, that is

$$dX_t = \tilde{\mu}dt + \sigma dW_t$$

Where $\tilde{\mu} = \mu - \frac{1}{2}\sigma^2$. We will price and work with the stock by looking at the evolution of X_t due to the benefits of working with an additive update function for later steps and the convenient fact that log-Euler is exact for GBM. Another helpful observation is that by this definition, $X_0 = 0$ always. This is helpful in simplifying later equations to a degree. After discretizing, we get the following

$$X_{i+1} = X_i + \tilde{\mu}(t_{i+1} - t_i) + \sigma\sqrt{t_{i+1} - t_i}Z_i$$

Where $Z \sim N(0, 1)$.

2.3 A model for the Auto-Callable Note

Given a collection of log-returns $(X_0, X_1, \dots, X_m)$, we can now work on defining the value of the structured product. Generally, we will have a set of coupons for each observed time period to be given in the event the note gets auto-called. We will represent these as Q_j . In the event of maturity, we will call a payout function $q(\frac{S_m}{S_{ref}})$ which can be equivalently expressed as $q(\frac{S_0}{S_{ref}}e^{X_m})$. The barrier itself is generally defined as a multiple of some reference price, generally the price on issuance. We will represent the multiple as B and the reference price as S_{ref} . This makes our barrier price BS_{ref} . Now, since we are tracking the value of X_t , we can adjust the barrier due to the fact that $S_t < BS_{ref}$ is an equivalent condition to $X_t < \ln(B\frac{S_{ref}}{S_0})$. For the sake of convenience, we will define this new barrier as $\gamma := \ln(B\frac{S_{ref}}{S_0})$. Now we can define the pricing function Q as follows

$$Q(X_0, X_1, \dots, X_m) = \begin{cases} e^{-r(t_j - t_0)}Q_j & \text{if } X_i < \gamma \leq X_j; \forall i < j \\ e^{-r(t_m - t_0)}q(\frac{S_0}{S_{ref}}e^{X_m}) & \text{if } X_i < \gamma; \forall i \end{cases} \quad (1)$$

This definition is nothing but the definition posited in [Alm+13] applied to the variable X_t as opposed to S_t . Note that $S_0e^{X_m} = S_m$

3 Truncation and One Step Survival in the Log Space

The usage of pure Monte Carlo with the above evaluation will not result in favorable results. The main issue here is the auto-call feature and the way it introduces discontinuities in the valuation. When a coupon gets autocalled, it instantly changes in value to whatever the coupon is at the time of the auto-call. These discontinuities introduce noise in the value estimation and have a drastic effect on first-order sensitivities. One Step Survival aims to solve this by generating paths that always remain below the barrier price and simultaneously tracking the probability of auto-call in each time period. By combining these two pieces of information, we can create a smooth valuation function that lacks the discontinuities of the previous approach while maintaining accuracy. We use the random variable Z_j in order to ensure our paths are well-behaved by truncating the normal distribution from which Z_j is sampled via inverse CDF sampling. Given

$$X_{i+1} = X_i + \tilde{\mu}(t_{i+1} - t_i)t + \sigma\sqrt{t_{i+1} - t_i}Z_i$$

Since an X_i value at or greater than γ will cause a knockout. Any Z_i value at or above $\frac{\gamma - X_i - \tilde{\mu}(t_{i+1} - t_i)}{\sigma\sqrt{t_{i+1} - t_i}}$ will cause the coupon to knockout on the next observation date. This means the probability of survival is given by $\mathbf{P}[Z_j \leq \frac{\gamma - X_i - \tilde{\mu}(t_{i+1} - t_i)}{\sigma\sqrt{t_{i+1} - t_i}}]$. This is exactly $\Phi(\frac{\gamma - X_i - \tilde{\mu}(t_{i+1} - t_i)}{\sigma\sqrt{t_{i+1} - t_i}})$ where Φ is the standard CDF of the normal distribution. We will define this quantity as p_j . Note, Φ and Φ^{-1} are both easily differentiable; this is a fact we are going to revisit soon.

We will use this value p_j in order to truncate our random variable sampling. This is the exact mechanism by which we ensure the path remains below the barrier price. If we sample

$$Z_i = \Phi^{-1}(p_i u_i); u_i \sim U[0, 1]$$

Then we can guarantee that the next value of X_t will remain below the barrier. Note, we also get the probability of survival, and therefore knockout, simultaneously. This is what allows us to define the smooth valuation function.

$$Q(X_0, X_1, \dots, X_m; p_0, p_1, \dots, p_{m-1}) = e^{-r(t_m - t_0)} L_m q\left(\frac{S_0}{S_{ref}} e^{X_m}\right) + \sum_{i=0}^{m-1} e^{-r(t_i - t_0)} L_i (1 - p_i) Q_i$$

Where $L_k = \prod_{n=0}^{k-1} p_n$ represents the probability of a stock price path surviving up until the period t_j . Note that, $L_0 = 1$. Intuitively, this formula is telling us that the final valuation is the probability of surviving until maturity times the value on maturity plus the coupon on each knockout times the probability of knockout *in that specific period*, which is itself the probability of survival until that period times the probability of knockout at that period. Now we can see that we've created an effective estimator that can be dealt with given *one particular realization of the stock price path*. Now there are two major questions: How do we go about carrying out a Monte Carlo simulation, and how do we go about differentiating the mechanism we create for valuation?

4 Vectorization Estimation and Sensitivities via Automatic Ad-joint Differentiation

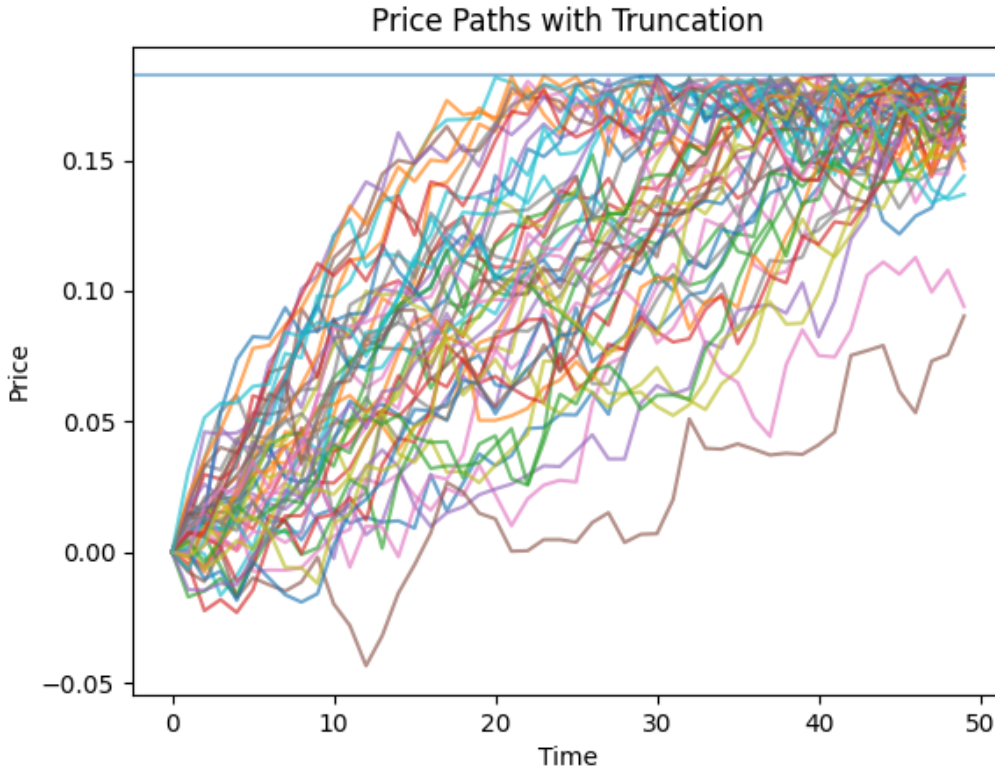


Figure 2: Visual of Price Paths with Truncation

In [Alm+13], the continuation of the single underlier case sees a procedural approach to Monte Carlo estimation followed by finite-difference estimation of the derivative. We will instead simulate the movements of stock prices with one vector $\hat{\mathbf{X}}_t = \langle X_{t,1}, X_{t,2}, \dots, X_{t,N} \rangle$ where N is the number of paths to sample. By making simple changes to the discrete update equation, we can vectorize our calculations. The first main

advantage of this is the massive upgrade in speed due to hardware-acceleration for linear algebra on most modern CPUs/GPUs. The second main advantage is the relative ease of creating a DCG(differentiable computational Graph) that we can apply Automatic Adjoint Differentiation to. This will give us a methodology that is faster and more accurate.

4.1 Vectorizing

As stated before, we are now dealing with a vector $\hat{\mathbf{X}}_t = \langle X_{t,1}, X_{t,2}, \dots, X_{t,N} \rangle$ and will rewrite the update equations to function on this vector.

$$\hat{\mathbf{X}}_{i+1} = \hat{\mathbf{X}}_i + \tilde{\mu}(t_{i+1} - t_i) + \sigma\sqrt{t_{i+1} - t_i}\Phi^{-1}(\hat{\mathbf{P}}_i \circ \hat{\mathbf{U}}_i)$$

Where

$$\hat{\mathbf{P}}_i = \Phi\left(\frac{\gamma - \hat{\mathbf{X}}_i - \tilde{\mu}(t_{i+1} - t_i)}{\sigma\sqrt{t_{i+1} - t_i}}\right)$$

$$\hat{\mathbf{U}}_j \sim U[0, 1]^N$$

Where $\circ$ represents the Hadamard product. We can now define two matrices, $\mathbf{X} \in \mathbf{R}^{m \times N}$ and $\mathbf{P} \in \mathbf{R}^{(m-1) \times N}$ and the valuation engine is a function on these matrices. importantly, $\mathbf{P}$ is a function on $\mathbf{X}$. This gives us Q as $\mathbf{Q} : \mathbf{R}^{m \times N} \rightarrow \mathbf{R}$. From here, Q functionally will work the same as an estimator, but it will iterate only over observation dates(which are generally not beyond the order of $\approx 10^2$ in number) as opposed to both time and samples(which is generally on the order of $\approx 10^6$).

This kind of estimator, with many inputs and one output, is optimal for AAD.

4.2 Automatic Adjoint Differentiation on Q

In addition to making our calculation faster, AAD will also provide a massive increase in stability for first and second-order sensitivities. Crucially for our purposes, the nature of DCGs allows us to carry out calculations regarding structural sensitivities as well. Note, this is the reason why we chose to write our estimator as a function of X_t . It increases linearly and does not cause any vanishing gradient issues in the DCG during the backward pass for AAD. One can imagine AAD as a generalisation of the "Backprop" algorithm used for Multi-layer perceptrons in machine learning. lets say I have a value $V = Q(\mathbf{X}, \mathbf{P})$ which represents my estimation of the value of a structured product. Now, I may want to estimate the sensitivity to any of the variables S_0, σ in order to calculate standard Greeks like Delta and Vega. I may even want to calculate structural sensitivities to variables like B or any of the coupons Q_j . This can also be done due to the flexibility of AAD. Additionally, since AAD allows for the calculation of Hessians, we can calculate various second-order sensitivities. More on this in the conclusion.

The evaluation function can also be written in a vectorized form, given below:

$$\hat{Q}(\hat{\mathbf{X}}_1, \hat{\mathbf{X}}_2, \dots) = \frac{1}{N} \mathbf{1}^\top (e^{-r(t_m - t_0)} \hat{\mathbf{L}}_m \circ q(\frac{S_0}{S_{ref}} e^{\hat{\mathbf{X}}_m}) + \sum_{i=0}^{m-1} e^{-r(t_i - t_0)} \hat{\mathbf{L}}_i \circ (1 - \hat{\mathbf{P}}_i) Q_i)$$

Quick notation note, here I use $\mathbf{1}^\top v$ as a shorthand for the sum of the elements of a vector v since it represents a dot product with a vector of all ones.

Mathematically speaking, we can visualize data flowing forward through the estimator. AAD uses the chain rule from multivariable calculus to come up with accurate expressions for the sensitivity to each variable. That is, for some general input variable κ I can find $\partial_\kappa V$ by expanding out to $\partial_\kappa \mathbf{X}(\partial_{\mathbf{X}} V + \partial_{\mathbf{X}} \mathbf{P} \partial_{\mathbf{P}} V)$. Note that this comes from the expansion of $\partial_\kappa \mathbf{X} d_{\mathbf{X}} V$ where d represents a total derivative. Continuing to expand this derivative would yield an expression of the form

$$\sum_{paths} (d_{X_m} V \partial_{\kappa} X_m + \sum_j \partial_{p_j} V \partial_{\kappa} p_j)$$

Critically, as mentioned before, both Φ and Φ^{-1} have well-defined derivatives in terms of the Gaussian PDF. One powerful example of this can be found in the pathwise derivative,

$$\partial_{p_j} X_{j+1} = \sigma \sqrt{(t_{i+1} - t_i)} \frac{u_j}{\phi(\Phi^{-1}(p_j u_j))}$$

Here, ϕ represents the Gaussian PDF. We can see that since ϕ is never zero, this particular derivative is somewhat well-behaved. This keeps this product well-behaved as it expands. See below for a simplified abstract diagram of the forward and backward passes through the DCG.

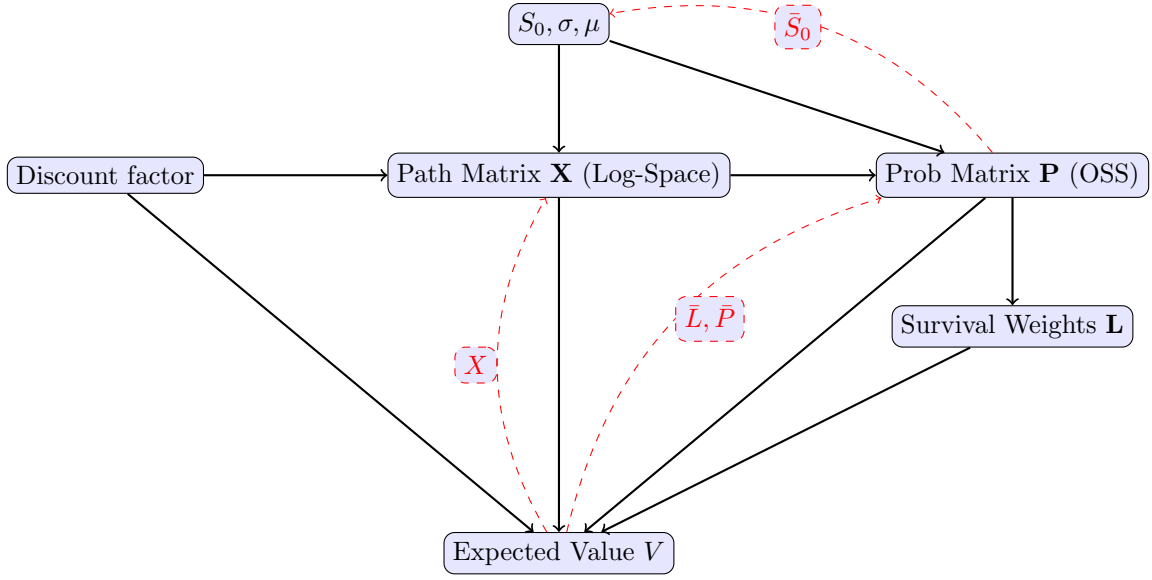


Figure 3: The Differentiable Computational Graph. Solid lines indicate the forward pass (Pricing), while dashed red lines indicate the adjoint pass (Sensitivities).

5 Results

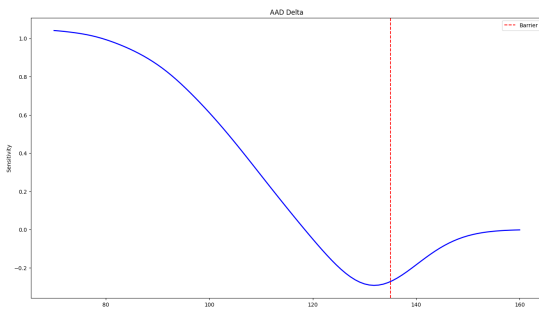
Here, we will visualize the stability of various sensitivities and compare it to the stability for a vanilla MC method. This will be done via visualization in this work, and more numerical results will be appended to this section soon as it is a working document. Crucially, all work will be done on one specific structured product to avoid bias. The terms of this product are given here. $S_0 = 3500, r = 0.04, \sigma = 0.2, ts = [1, 2], Qs = [110, 120], q(s) = s \times 100, B = 1.0, S_{ref} = 4000$.

Where the results are truly remarkable is the difference in the amount of noise found in the first and second-order sensitivities. Worthy of note is the fact that, due to vectorization, the method shown in this paper is much faster and more efficient than a procedural, vanilla Monte Carlo-based approach.

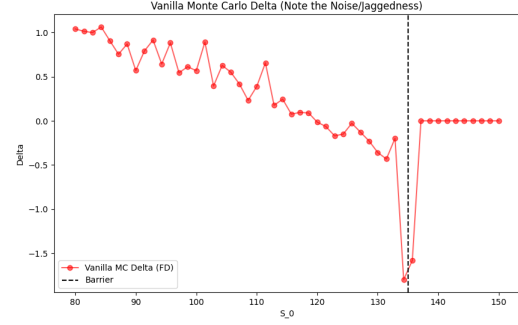
Table 1: Numerical Performance Comparison: Value and Delta Estimators

Methodology	Value (Mean)	Value (Std)	Delta (Mean)	Delta (Std)
Procedural Monte Carlo	87.1931	20.6385	0.0236	1.4347
MC + OSS	87.1275	9.9426	0.0192	0.6999
MC + OSS + JAX/AAD	87.1732	9.9613	0.0203	0.0097

Note: The AAD implementation provides significant variance reduction for sensitivities, decreasing the standard deviation for Delta by approximately $72\times$ compared to the MC+OSS implementation and $147\times$ relative to the vanilla procedural Monte Carlo.

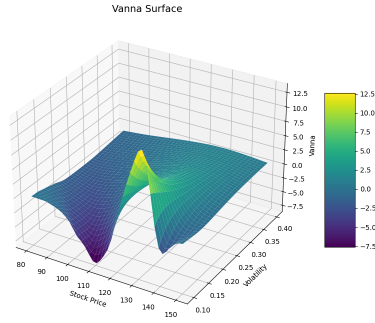


(a) AAD (Proposed)

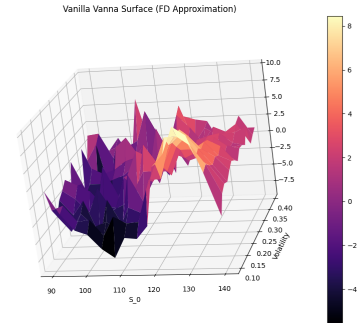


(b) Vanilla MC (Benchmark)

Figure 4: Delta and Gamma comparison. Note the suppression of numerical noise in the AAD implementation (a) compared to the discrete jumps and instability in the standard Monte Carlo approach (b).



(a) AAD (Proposed)



(b) Vanilla MC (Benchmark)

Figure 5: Vanna comparison for both models. Note the suppression of numerical noise in the AAD implementation.

6 Conclusion and Analysis

All in all, this methodology provides a robust, efficient, flexible way to calculate the Greeks of a structured product. As mentioned in [Pro25], structured products are a growing part of the European market and the world of finance as a whole. They offer unique opportunities for the buy side, sell side, and regulatory agencies alike. However, the degree to which agencies will be able to capitalize on these instruments is directly tied to how reliable and accurate their statistics are. The Greeks have long dominated the world of finance as a "genetic profile" of an instrument. By providing a robust methodology here for the calculation

of these Greeks we hope to aid investors, sellers, and regulators. For example, the code used to generate statistics and plots for this paper can easily be turned into a CLI or GUI that can be in the hands of every Quant trader or Quant researcher who deals with these products. The ability of this methodology to define second-order Greeks with such stability and reliability makes this a potent tool for regulators to understand the strengths and weaknesses of each product. For example, note the presence of the Vanna ridge in 5; this represents the high hedging risk as the underlier approaches the barrier. The ability to calculate "structural Greeks", like sensitivity to Barrier price, also makes this a powerful tool for those on the sell side while they are designing their products.

For anyone even tangentially involved with the world of structured products, we hope to advance their ability to understand the instruments they are designing, trading, or regulating. With this system, hopefully, that can be done even more reliably.

References

- [Boy77] Phelim P Boyle. "Options: A monte carlo approach". In: *Journal of financial economics* 4.3 (1977), pp. 323–338.
- [GS01] Paul Glasserman and Jeremy Staum. "Conditioning on one-step survival for barrier option simulations". In: *Operations Research* 49.6 (2001), pp. 923–937.
- [Alm+13] Thomas Alm et al. "A Monte Carlo pricing algorithm for autocallables that allows for stable differentiation". In: *Journal of Computational Finance* 17.1 (2013), pp. 43–70.
- [Pro25] Structured Retail Products. *SRP in Brief: all aboard the trans Europe express*. <https://www.structuredretailproducts.com/insights/82008/srp-in-brief-all-aboard-the-trans-europe-express>. Accessed: 2026-2-13. 2025.